

PROJECT HANDOFF GUIDE

SHINZI Games

인턴십 과제 프로젝트

Unity 탑다운 1:1 액션 게임 — 구현 구조·데이터 파이프라인·인계 가이드

문서 기준

2026-08-18 저장소의 실제 코드·ScriptableObject·Addressables 설정·씬 구성을 기준으로 작성했습니다.
현재 구현과 향후 확장안을 분리해 기술하며, 구현되지 않은 기능은 현재 기능처럼 표현하지 않습니다.

항목	내용
프로젝트 유형	Unity / 탑다운 1:1 액션 프로토타입
핵심 입력	WASD 이동, 마우스 에임·공격, 대시 입력
핵심 데이터 흐름	Excel → 검증 → ScriptableObject → 런타임 초기화
리소스 로딩	Addressables 기반 플레이어·AI·무기·픽업·투사체 로드
맵 운용	현재 1 개 맵을 씬에 고정 배치

이 문서는 인계받는 사람이 게임 흐름과 클래스 책임을 빠르게 파악하고, 데이터 변경·리소스 추가·오류 점검을 재현할 수 있도록 구성했습니다.

문서 구성

구분	확인할 내용
1. 게임 설명	플레이 목표, 조작, 승패·진행 규칙
2. 전체 설계 구조	레이어 구조, 런타임 시작·종료 흐름
3. 역할 분리	Manager·Controller·Brain·Component·Data 의 책임
4. 전투·AI·무기	공통 캐릭터 기능, AI 판단, 무기 런타임 구조
5. 데이터 도구	Excel→SO 2-Pass 변환, 검증, 참조 관계
6. Addressables	주소 저장, 비동기 생성, 해제·버전 보호
7. 확장·인계	현재 제약, 확장 지점, 제출 전 점검

표기 원칙

[현재]는 저장소에 구현된 동작, [확장]은 요구가 생겼을 때 적용할 설계 방향, [점검]은 제출·인계 전에 확인해야 할 항목을 뜻합니다.

1. 게임 설명

플레이어가 제한 시간 동안 한 명의 AI 와 전투하는 탑다운 1:1 액션 게임입니다. 맵에 생성되는 무기를 획득해 교체할 수 있고, 승리 횟수에 따라 다음 매치에서 사용할 AI 와 드롭 구성이 달라집니다.

핵심 플레이

- 고정 카메라 환경에서 WASD 로 이동하고 마우스 위치를 향해 회전·조준합니다.
- 공격과 대시를 사용할 수 있으며, 공격·대시 쿨다운은 HUD 에 표시됩니다.
- 맵에 생성된 다른 무기를 획득하면 현재 무기 데이터와 손에 표시되는 무기 프리팹이 교체됩니다. 같은 무기는 다시 장착하지 않습니다.

- AI 는 사용 가능한 무기를 우선 탐색하고, 무기가 없거나 목표 무기가 없으면 플레이어를 추격합니다. 장착 무기의 공격 범위 안에서는 공격합니다.

승패와 진행

상황	결과
AI 체력 0	플레이어 승리, 누적 승수 +1
플레이어 체력 0	플레이어 패배
제한 시간 종료 시 AI 생존	무승부 없이 플레이어 패배
다음 매치 시작	현재 승수 이하의 MinWins 중 가장 높은 MatchData 선택
재시작	승수를 0 으로 초기화하고 첫 매치 규칙으로 시작

공격 중 이동 규칙

현재 구현에서는 `WeaponType.Range` 인 무기가 공격 중일 때 이동을 막고, 근접 무기는 이동과 공격을 동시에 허용합니다. 무기 종류와 행동 규칙의 결합을 줄이려면 향후 `blocksMovementWhileAttacking` 같은 능력 값으로 치환할 수 있습니다.

2. 전체 설계 구조

구조의 핵심은 입력·판단·실행·데이터·표현을 한 클래스에 모으지 않고, 변경 이유가 다른 책임을 분리한 것입니다. Controller 는 흐름을 조정하고 실제 동작은 각 기능 컴포넌트가 소유합니다.

설계 구조도



의존 방향은 위에서 아래입니다. 하위 기능은 UI 나 매치 화면을 직접 알지 않으며, UI 는 전투 로직을 실행하지 않습니다.

런타임 매치 흐름

1. 시작 버튼 → MatchManager.StartGame()이 현재 승수에 맞는 MatchData 를 선택합니다.
2. MatchData 가 참조하는 AIData 와 PlayerData 주소로 플레이어와 AI 를 순차 생성합니다.
3. 생성된 인스턴스에 데이터·타겟·WeaponSpawner 를 Initialize()로 주입합니다.
4. WeaponSpawner 가 드롭 테이블과 생성 주기를 적용하고 MatchStarted 이벤트로 HUD 를 연결합니다.

5. 플레이어 입력과 AI 판단이 각 Controller 를 거쳐 Movement·Aim·Combat 에 전달됩니다.
6. 사망 또는 시간 종료 → 입력·AI 제어 정지 → 무기 제거 → 캐릭터 Addressables 해제 → MatchEnded 이벤트로 결과 UI 표시.

비동기 시작 순서를 순차화한 이유

AI 초기화에는 플레이어 Transform 이 필요하고, HUD 와 무기 스폰은 두 캐릭터 초기화가 끝난 뒤 시작해야 합니다. 따라서 플레이어 → AI → 스폰·이벤트 순으로 완료 지점을 명확히 두었습니다.

3. 역할 분리와 책임

구분	대표 클래스	책임	책임 밖
Manager	MatchManager	매치 전체 생명주기·승수·승패·생성/해제	이동 계산, 무기 판정, UI 렌더링
Controller	PlayerController, AIController	한 캐릭터의 입력/판단 결과를 기능에 전달	기능 내부 계산과 데이터 원본 소유
Brain	AIBrain	상황을 AIState 와 CurrentTarget 으로 결정	NavMesh 이동, 회전, 공격 실행
Component	Movement, Aim, Combat, Health	한 가지 런타임 기능과 상태 소유	매치 진행, 화면 전환
Runtime	WeaponRuntime 계열	장착된 무기의 공격 방식 구현	누가 장착할지·드롭 확률 결정
Data	PlayerData 등 SO	밸런스·주소·참조 데이터 제공	매 프레임 행동 수행
View/UI	WeaponView, MatchUI 계열	표현과 사용자 피드백	게임 규칙의 최종 판단

플레이어 책임 분리

클래스	역할
InputReader	Unity Input System 의 Move·Aim·Attack·Dash 를 읽고 입력 상태만 보관합니다.
PlayerController	Aim → Attack → Dash → Move 순서로 기능을 조정하고, UI 에 필요한 읽기 전용 상태를 제공합니다.
PlayerMovement	CharacterController 이동, 중력, 대시·쿨다운, 넉백을 처리합니다.
PlayerAim	마우스 스크린 좌표를 수평 월드 지점으로 변환해 캐릭터 회전을 적용합니다.
CharacterCombat	현재 무기의 공격 가능 여부·쿨다운·공격 중 이동 차단 여부를 관리합니다.

PlayerController 가 모든 기능을 직접 구현하지 않는 이유

플레이어 입력을 전투·이동 코드 안에 직접 넣으면 AI 가 같은 전투·체력·무기 기능을 재사용하기 어렵습니다. Controller 는 ‘언제 호출할지’를 조정하고, 기능 컴포넌트는 ‘어떻게 동작할지’를 담당하므로 입력 변경이 전투 구현을 흔들지 않습니다.

4. 캐릭터·AI·무기 설계

공통 캐릭터 기능

클래스	핵심 책임
CharacterHealthSystem	최대/현재 체력, TakeDamage(), HealthChanged(current,max), Died 이벤트
CharacterDamageReceiver	DamageInfo 수신 후 체력 감소, IKnockbackReceiver 에 넉백 전달
CharacterWeaponHolder	현재 WeaponData 소유, 장착 가능 여부·교체·WeaponChanged 이벤트
CharacterWeaponView	WeaponChanged 를 구독해 손 소켓에 무기 프리팹 생성, WeaponRuntime 초기화·이전 무기 해제
CharacterAnimation	이동 속도로 Idle/Walk 전환, 사망 트리거 처리

AI 구조

현재 AI 는 한 마리를 기준으로 하며, 복잡한 행동 트리가 필요한 수준이 아니므로 enum 기반 상태 판단을 사용합니다. AIBrain 은 일반 C# 클래스로 판단만 수행하고, AIController 가 실제 기능을 호출합니다.

상태	진입 조건·동작
Idle	플레이어 타겟이 없을 때 정지
SeekWeapon	장착 가능한 활성 무기 픽업이 있으면 가장 가까운 픽업으로 이동
Chase	무기가 없거나 공격 범위 밖이면 플레이어 추격
Engage	무기 Range 안에서 플레이어를 바라보고 공격; 원거리 공격 중에는 정지
Dead	판단과 이동 중단

AIMovement 는 NavMeshAgent 로 경로를 탐색하고, 넉백은 agent.Move()에 감쇠 벡터를 적용합니다. 향후 맵에 장애물이 추가되어도 베이킹된 NavMesh 경로를 사용할 수 있다는 점이 선택 근거입니다.

무기 런타임 구조

타입	처리 방식
WeaponRuntime	WeaponData·Owner·공격 상태를 보유하는 추상 기반 클래스
HitBoxWeaponRuntime	공격 시간 동안 Collider 를 켜고, 한 번의 공격에서 같은 대상 중복 타격을 HashSet 으로 방지
ProjectileWeaponRuntime	ProjectileData 주소로 투사체 프리팹을 생성하고 방향·소유자·데미지 데이터를 전달
Projectile	투사체 공통 초기화, 소유자 충돌 제외, 수명 종료와 Addressables 해제
StraightProjectile	Rigidbody 속도로 직선 이동하고 대상 또는 비 Trigger 장애물 충돌 시 해제

Rigidbody 를 투사체에 둔 이유

플레이어 이동은 직접 제어가 우선이므로 CharacterController 가 적합하고, 투사체는 물리 업데이트·Trigger 충돌이 핵심입니다. 필요한 객체에만 Rigidbody 를 두어 플레이어·AI 의 이동 안정성과 투사체 충돌 검출을 분리했습니다.

5. 데이터 구조와 Excel → SO 도구

밸런스와 리소스 주소를 Excel 에서 관리하고, 에디터 도구가 ScriptableObject 로 변환합니다.
런타임은 Excel 을 직접 읽지 않고 생성된 SO 만 참조합니다.

테이블 참조 관계

상위 데이터	참조	용도
MatchData	AIData	해당 승수 구간에서 생성할 AI
MatchData	MatchDropData	해당 매치의 무기 드롭 가중치 목록
AIData	AIBehaviorData	반응 시간 등 AI 판단 성향
MatchDropData.Entry	WeaponData	드롭 대상 무기와 가중치
WeaponData	ProjectileData (선택)	원거리 무기의 투사체 속성·프리팸 주소
PlayerData	독립 데이터	플레이어 체력·속도·대시·프리팸 주소

관계 요약: MatchData → AIData → AIBehaviorData / MatchData → MatchDropData → WeaponData → ProjectileData

2-Pass 변환 파이프라인

1. 읽기

ExcelReader: ExcelDataReader 로 워크북을 열고 DataTable 로 변환

↓ 스키마·ID·값 검증

2. 기본 필드 적용 (Import)

DataTableValidator + ExcelValueConverter + DataImporter: 문자열·숫자 등 기본 필드로 SO 생성/갱신

↓ 대상 SO 가 모두 존재한 뒤

3. 참조 연결 (Resolve)

DataReferenceValidator + DataImporter: ID 로 대상 SO 를 찾아 일반 필드·List 요소 참조 적용

↓ 최상위 작업 종료 시 1 회

4. 저장

DataImporterMenu: 전체 성공 여부를 모아 AssetDatabase.SaveAssets() 호출

도구별 책임

클래스	역할
ExcelReader	파일 스트림과 reader 를 using 으로 닫고 DataTable 반환
DataTableValidator	일반 테이블의 빈/중복 ID, 리스트 테이블 ID, 헤더·기본 값 검증
ExcelValueConverter	셀 문자열을 대상 FieldInfo 타입으로 변환
DataReferenceValidator	일반·List 의 SO 참조가 모두 유효한지 먼저 검증하고 실패 시 부분 참조 적용 방지
DataImporter	검증된 값을 SO 에 반영하고 기존 SO 는 갱신하여 GUID 와 참조 유지
DataImporterMenu	개별/전체 Import·Resolve 메뉴, 성공 결과 집계, 최종 저장

Import 와 Resolve 를 분리한 이유

Excel 의 참조 대상 SO 가 아직 만들어지지 않은 순서 문제를 제거하기 위해서입니다. 1-Pass 에서 모든 SO 의 기본 형태를 확보하고, 2-Pass 에서 ID 참조를 연결하므로 테이블 순서에 대한 결합이 줄고 참조 오류를 명확하게 보고할 수 있습니다.

데이터 변경 절차

1. Excel 의 필드 헤더와 대상 SO 직렬화 필드 이름을 일치시킵니다.
2. 주소·ID·가중치·수치 값을 수정하고 파일을 저장합니다.
3. 한 테이블만 바뀌었다면 Tools/Excel Import/Individual 메뉴를 사용합니다.
4. 여러 참조 테이블이 함께 바뀌었다면 0. Import And Resolve (전체)를 실행합니다.
5. Console 의 검증 오류가 없는지 확인한 뒤 생성된 SO 참조와 실제 플레이를 점검합니다.

6. Addressables 운용

Addressables 는 매치마다 달라지거나 데이터 주소로 선택되는 프리팹을 런타임에 생성하는 데 사용합니다. UI 에 고정된 이미지나 프리팹 내부 종속 에셋까지 무조건 개별 주소화하지 않습니다.

현재 등록·사용 자원

분류	주소 예	실제 사용 지점
플레이어	Characters/Player	MatchManager 가 PlayerData 주소로 생성
AI	Characters/AI_Easy, AI_Normal, AI_Hard	MatchData→AIData 주소로 생성
손 무기	Weapons/Sword, Hammer, Shield, Bow	CharacterWeaponView 가 장착 변경 시 생성
무기 픽업	Pickups/WeaponPickup	WeaponSpawner 가 AssetReferenceGameObject 로 생성
투사체	Projectiles/Arrow	ProjectileWeaponRuntime 이 ProjectileData 주소로 생성

비동기 수명주기

단계	처리
요청	SO 에 저장된 주소 또는 AssetReference 로 InstantiateAsync 호출
완료 검증	Status, 컴포넌트 존재, Initialize() 성공 여부 확인
경합 방지	matchVersion:spawnVersion-view version 으로 이전 비동기 결과를 폐기
정상 해제	생성 인스턴스는 Addressables.ReleaseInstance()로 반환
실패 해제	실패 handle 은 Addressables.Release(handle) 처리

버전 검사의 목적

비동기 로드 완료 전에 매치가 종료되거나 다른 무기로 교체될 수 있습니다. 요청 시점의 버전과 완료 시점의 버전이 다르면 늦게 도착한 결과를 즉시 해제하여 이전 매치·이전 무기가 현재 상태를 덮어쓰지 못하게 합니다.

맵 운용 정책

[현재] 씬 고정 맵

현재 요구사항은 맵 1 개이며 Match_1 프리팹 인스턴스, Player/AISpawnPoint, WeaponSpawner 와 무기 스폰 지점, 베이킹된 NavMeshSurface 를 플레이 씬에서 사용합니다. 따라서 MatchManager 가 맵을 Addressables 로 로드하지 않습니다.

[확장] 다중 맵

맵이 추가되면 MatchData 에 맵 주소를 추가하고 MapLoader 또는 MapManager 가 프리팹을 생성하도록 확장합니다. 맵 프리팹의 MapContext 가 Player/AISpawnPoint, WeaponSpawnPoints, NavMeshSurface 를 외부에 제공하면 씬 참조 단절을 막을 수 있습니다.

7. 주요 설계 선택과 근거

선택	적용	선택 근거
플레이어 이동	CharacterController	정교한 연쇄 물리보다 직접 제어·충돌·넉백 재현성이 중요
AI 이동	NavMeshAgent	현재 맵과 향후 장애물 맵에서 경로 탐색을 재사용하고 난이도별 판단과 이동을 분리
AI 판단	enum 상태 + AIBrain	현재 상태 수와 AI 1 마리 규모에 맞고, 완전한 State 패턴·Behavior Tree 의 객체/설정 비용을 피함
공통 전투	기능 컴포넌트 공유	플레이어 입력과 AI 판단이 달라도 Health·Combat·Weapon 기능은 동일하게 재사용
데미지 전달	DamageInfo 구조체	한 번의 타격에 함께 이동하는 값 묶음이며 독립 생명주기·상속이 필요하지 않음
UI 갱신	이벤트 + 읽기 상태	체력·매치 전환은 이벤트, 연속 쿨다운은 HUD 가 읽어 갱신해 불필요한 결합을 줄임
데이터	Excel + SO	기획 데이터 편집성과 Unity 런타임 참조 안정성을 동시에 확보
리소스	선택적 Addressables	데이터로 바뀌는 프리팹은 주소 로드, 항상 함께 필요한 종속 에셋은 프리팹 참조 유지

상태머신과 Behavior Tree 를 지금 사용하지 않은 이유

플레이어는 이동·에임·공격이 동시에 일어나므로 하나의 단일 상태로 제한하면 조합 상태가 급격히 늘어납니다. 반면 AI 는 SeekWeapon·Chase·Engage 처럼 우선 행동을 하나 선택해야 하므로 상태 구분이 유효합니다. 현재 판단은 선형 우선순위로 충분하고, 조건 조합·병렬 행동·서브트리 가 크게 늘 때 Behavior Tree 도입을 검토합니다.

Manager 를 제한적으로 사용한 이유

전역 접근이 필요한 클래스마다 Manager 를 만들지 않았습니다. MatchManager 만 매치 전체 생명주기를 소유하고, WeaponSpawner 는 무기 생성이라는 장면 내 책임에 한정합니다.

Addressables 서비스나 오브젝트 풀은 중복 로딩 정책.캐시.풀링 요구가 실제로 생길 때 분리하는 편이 현재 규모에서 책임과 호출 경로를 더 명확하게 유지합니다.

8. 인계 및 운영 가이드

씬 필수 구성

항목	필수 연결
MatchManager	MatchData 목록, PlayerData, WeaponSpawner, Player/AISpawnPoint, deathAnimationDuration
WeaponSpawner	WeaponPickup AssetReference, 무기 SpawnPoints
플레이 맵	콜라이더, 베이킹된 NavMeshSurface, 이동 가능 영역
UI	GameStart/Match/Result Canvas 와 MatchUI·HUD·Result 연결
캐릭터 프리팹	Controller, Movement/Aim, Health/DamageReceiver, Combat, Holder/View, Animation
무기 프리팹	WeaponRuntime 파생 컴포넌트, HitBox 또는 Projectile spawn point

새 콘텐츠 추가 절차

추가 대상	작업 순서
AI	AI/Behavior Excel 행 추가 → 프리팹 Addressable 등록 → 주소 입력 → Import/Resolve → MatchData 참조 확인
근접 무기	WeaponData 행·프리팹·HitBoxWeaponRuntime 구성 → Addressable 등록 → DropList 가중치 연결
원거리 무기	ProjectileData·투사체 프리팹 추가 → ProjectileWeaponRuntime 무기 연결 → 두 주소 등록 → Import/Resolve
매치 단계	MatchData 에 MinWins·AI·DropList·시간 설정 → MatchManager 목록에 SO 추가
맵	현재는 씬 교체/수정; 다중 맵 요구 시 MapContext 기반 Addressable 로더부터 도입

제출·인계 전 점검

우선도	점검 항목	판정 기준
필수	Addressables Content Build	Player/AI/무기/픽업/투사체가 에디터와 빌드에서 모두 생성·해제
필수	전체 Excel Import + Resolve	빈·중복 ID, 타입 변환, 참조 오류 없이 SO 갱신
필수	게임 루프	시작→진행→승/패→다음/재시작이 반복 가능
필수	씬 참조	MatchManager·WeaponSpawner·스폰 포인트·UI·NavMeshSurface 연결
정리	Matches/Match_1 Addressable 등록	현재 씬 고정 정책이면 사용되지 않는 등록을 제거
정리	StraightProjectile Rigidbody	projectileRigidbody 필드를 실제 사용하거나 필드를 제거하여 중복 접근 제거
선택	WeaponType 의존	무기 행동 조합이 늘면 이동 차단을 capability bool 로 이전
제출	외부 무료 에셋 출처	프로젝트 내 별도 메모 또는 제출 문서에 출처 기록

현재 알려진 코드 정리 항목

StraightProjectile 은 projectileRigidbody 필드를 선언했지만 TryShoot()에서 GetComponent<Rigidbody>()를 다시 호출합니다. 기능 오류는 아니지만 필드를 캐시할 목적이었다면 해당 필드를 사용하고, 아니라면 선언을 제거해야 의도가 명확합니다. 또한 Matches/Match_1 은 Addressables 그룹에 남아 있으나 현재 MatchManager 에서 로드하지 않으므로 씬 고정 정책 확정 시 등록을 정리합니다.

9. 기능 검증 매트릭스

영역	검증 시나리오	기대 결과
입력	WASD·마우스·공격·대시	이동/에임 독립, 1 회성 입력 소비, 쿨다운 준수
전투	근접·활 공격	HitBox/투사체 데미지·넉백, 같은 공격 중 중복 타격 방지
원거리 이동	활 공격 중 이동 입력	공격 지속 중 정지, 종료 후 이동 복구
무기 교체	서로 다른 무기/같은 무기 픽업	다른 무기 교체, 같은 무기 무시, 이전 View 해제
AI	무기 있음/없음/범위 내외	SeekWeapon→Chase→Engage 전환, 플레이어 방향 갱신
스폰	복수 스폰 주기·실패	점유 위치 중복 방지, 실패 예약 복구, 종료 시 모두 해제
승패	AI 사망/플레이어 사망/시간 종료	승/패·승수·결과 UI 가 규칙대로 처리
데이터	개별/전체 Import	기존 GUID 유지, 잘못된 ID·참조 차단, 성공 시 한 번 저장
Addressables	매치 반복·빠른 교체	오래된 완료 콜백이 현재 상태를 덮지 않고 인스턴스 누수 없음

리뷰 인터뷰에서 설명할 핵심

- Excel 은 편집 원본, SO 는 Unity 런타임 데이터입니다. 2-Pass 로 생성과 참조 연결을 분리해 순서 문제와 부분 적용을 줄였습니다.
- Addressables 는 '모든 에셋'이 아니라 런타임에 데이터로 선택·교체되는 프리팹에 적용했습니다. 생성과 해제 책임은 요청한 클래스가 가집니다.
- 플레이어와 AI 의 차이는 입력/판단이며, 체력·전투·무기 기능은 공통 컴포넌트로 재사용합니다.
- 현재 요구사항에 맞게 enum AI 상태와 씬 고정 맵을 선택했고, 복잡도가 증가할 때의 Behavior Tree·MapContext 확장 지점을 남겼습니다.
- 예외 처리는 가능한 모든 상황을 막는 것이 아니라, 비동기 로드·데이터 무결성·인스턴스 해제처럼 실패 비용이 큰 경계에 집중했습니다.

인계 결론

프로젝트는 데이터 편집(Excel), Unity 데이터(SO), 리소스 로드(Addressables), 매치 흐름(MatchManager), 캐릭터 기능(Component), 표현(UI)을 분리한 구조입니다. 새 AI·무기·매치 데이터는 기존 파이프라인으로 추가할 수 있으며, 맵·풀링·사운드처럼 현재 요구 밖의 시스템은 필요 시 확장하도록 경계를 남겼습니다.

부록. 주요 파일 위치

영역	경로
매치	Assets/3.Scripts/Match/MatchManager.cs
플레이어	Assets/3.Scripts/2.Player/
AI	Assets/3.Scripts/3.AI/
공통 캐릭터	Assets/3.Scripts/1.Character/
무기	Assets/3.Scripts/Weapon/
UI	Assets/3.Scripts/UI/
런타임 데이터	Assets/3.Scripts/RunTime/
Excel 도구	Assets/Editor/Scirpts/ExcelTool/
Excel 원본	Assets/Editor/ExcelData/
생성된 SO	Assets/8.GameData/
Addressables 설정	Assets/AddressableAssetsData/
플레이 씬	Assets/1.Scenes/PlayerMatchScene.unity

주의: 폴더명 'Scirpts' 는 현재 저장소의 실제 경로 표기를 그대로 사용했습니다.